МИКРОСЕРВИСНАЯ АРХИТЕКТУРА

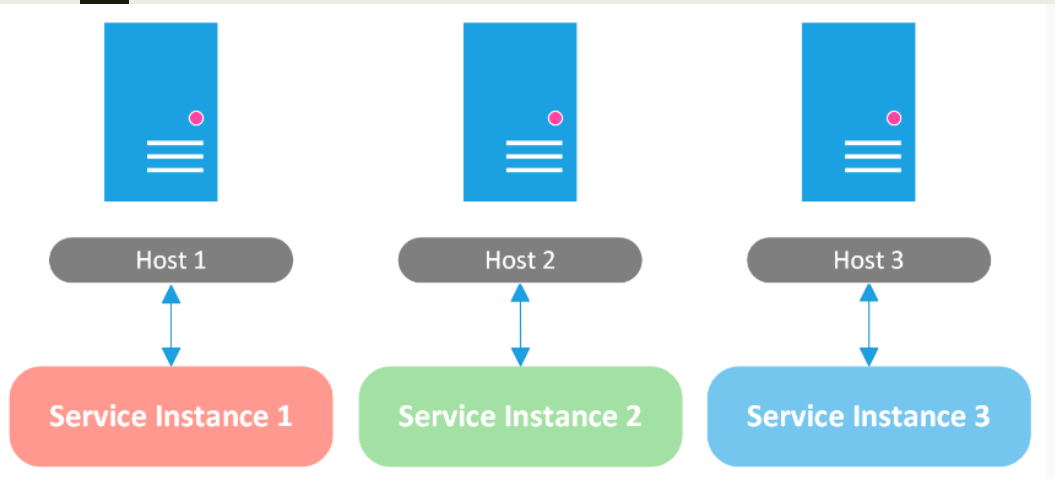
Заканчиваем паттерны, смотрим файловое хранилище и распределенную трассировку

Паттерны развертывания микросервисов

- Этот блок шаблонов описывает возможные варианты развертывания разработанных микросервисов.
- Микросервисы – это дорогая инфраструктура и по подъему и по обслуживанию.
- Поэтому необходимо всегда оценивать целесообразность ее построения. Но если уж надо строить, то эффективно.

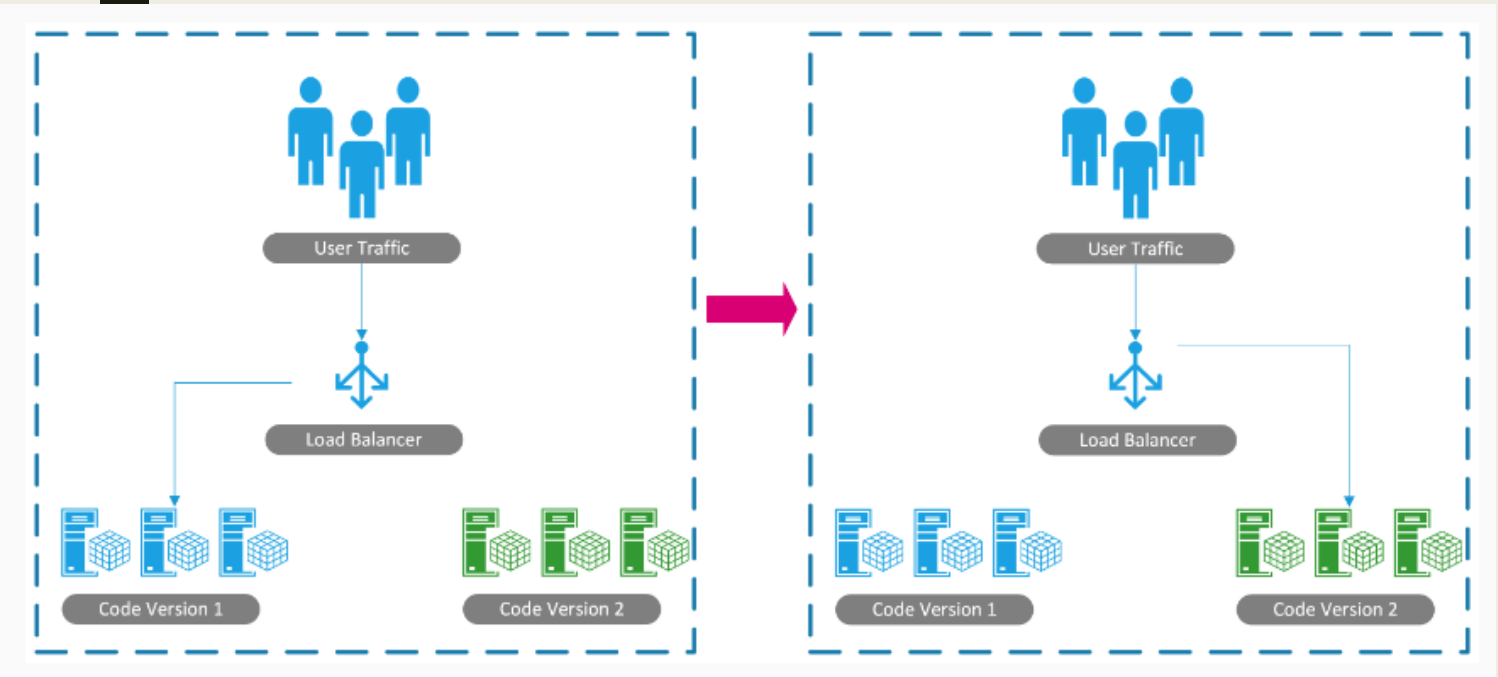
https://mcs.mail.ru/blog/26-osnovnyh-patternov-mikroservisnoj-razrabotki

Шаблон «Экземпляр сервиса на хост» (Service Instance Per Host)



- При переходе на микросервисную архитектуру рекомендуется проводить развертывание каждого экземпляра сервиса на собственном хосте (виртуальном или физическом). Паттерн Service Instance Per Host позволяет изолировать экземпляры сервисов друга от друга, избежать конфликтов версий и требований к ресурсам, максимально использовать ресурсы хоста, а также легче и быстрее проводить повторные развертывания. К недостаткам паттерна можно отнести потенциально менее эффективное использование ресурсов по сравнению с развертыванием нескольких экземпляров на хост.
- Иногда выделяют разновидности описанного шаблона, наиболее часто используемые на практике: «Экземпляр сервиса на виртуальную машину» (Service Instance Per VM) и «Экземпляр сервиса на контейнер» (Service Instance Per Container). При их использовании каждый экземпляр сервиса упаковывается и разворачивается в виде отдельной виртуальной машины либо контейнера соответственно.

Шаблон «Сине-зеленое развертывание» (Blue-Green Deployment)



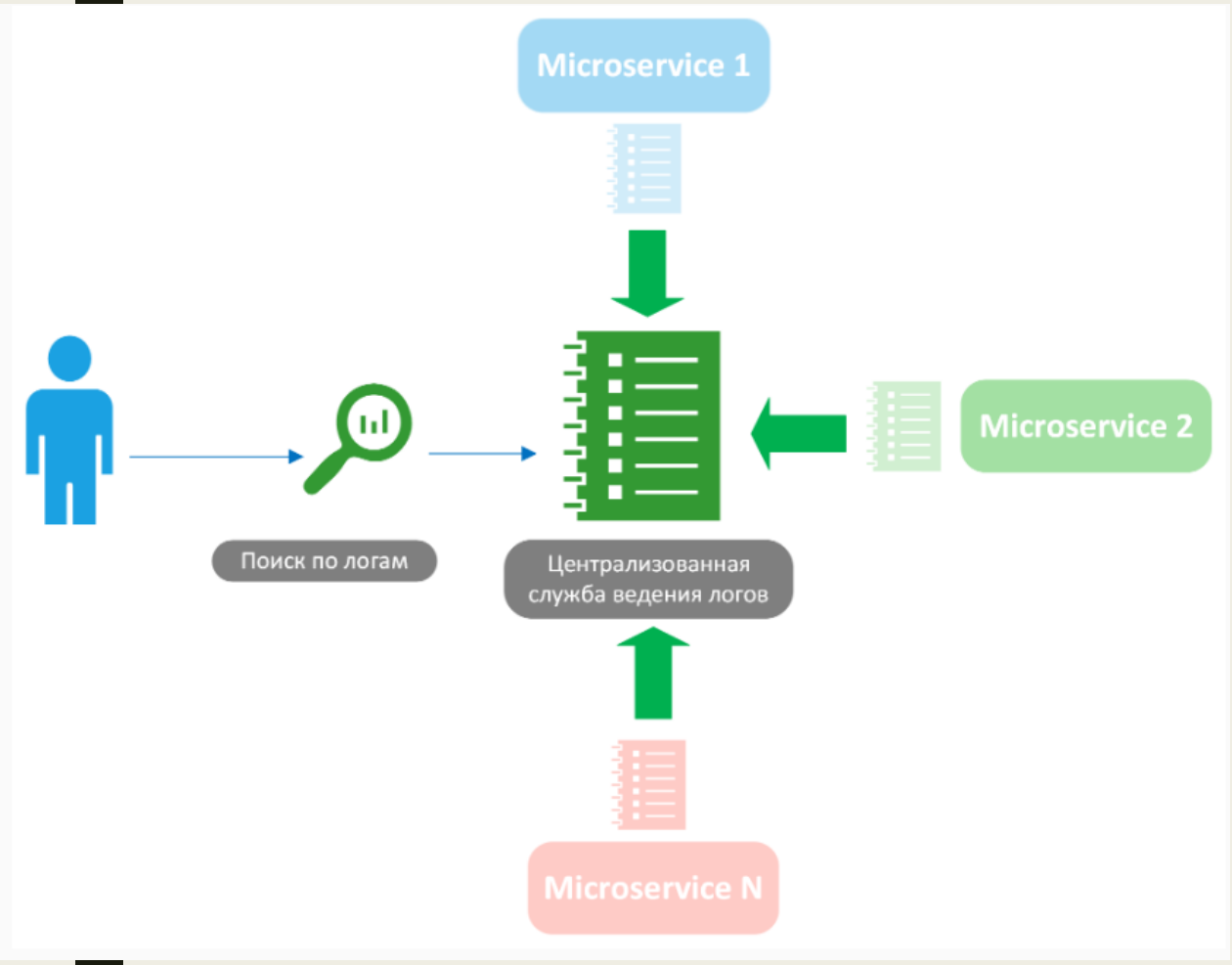
- Паттерн позволяет выполнить развертывание новых версий сервисов максимально незаметно для пользователей, сократив время простоя до минимума. Это достигается за счет запуска двух идентичных производственных сред — условно синего и зеленого цвета. Предположим, что синий — это существующий активный экземпляр, а зеленый — это новая версия приложения, развернутая параллельно с ним.

- В любой момент времени только одна из сред является активной, и именно она обслуживает весь производственный трафик. После успешного развертывания новой версии — с прохождением всех тестов и так далее — трафик переключается на нее. В случае ошибок всегда можно вернуться к предыдущей версии.

Паттерны мониторинга микросервисов

- За состоянием любой развернутой системы нужно наблюдать.
- Этот блок шаблонов охватывает возможные варианты построения мониторинга работы микросервисов.

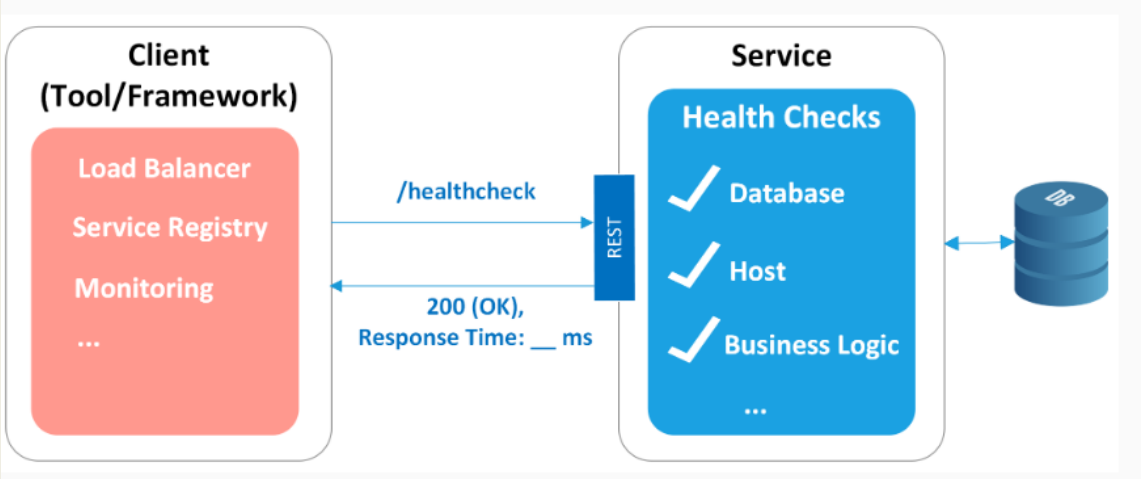
Шаблон «Агрегация логов» (Log Aggregation)



Хорошей практикой при разработке микросервисов считается ведение логов каждым экземпляром сервиса. Логи могут содержать ошибки, предупреждения, информационные и отладочные сообщения. Но с увеличением числа сервисов анализ логов, разнесенных по различным хостам, становится затруднительным.

Паттерн Log Aggregation предлагает использовать централизованную службу ведения логов, которая будет собирать логи от каждого экземпляра сервиса. Это предоставит пользователям единую точку для поиска, анализа логов и настройки предупреждений, которые будут запускаться при появлении в них определенных сообщений.

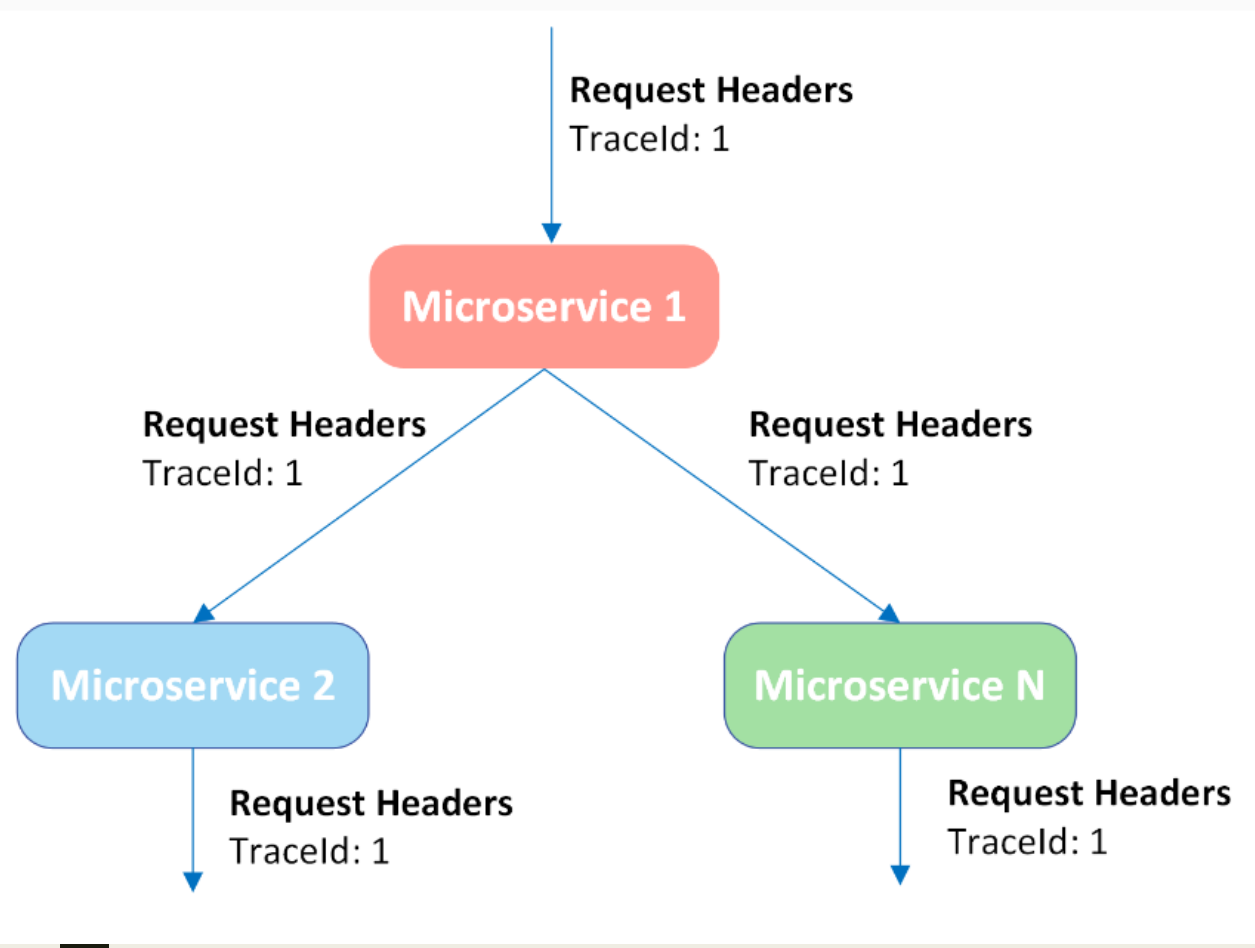
Шаблон «Проверки здоровья» (Health Check)



- Иногда экземпляр сервиса, более не способный обрабатывать внешние запросы, остается доступен для других подсистем. Например, сервис может исчерпать пул соединений к базе данных — фактически он становится неработоспособным, но принимать внешние запросы по-прежнему в состоянии, хоть и без последующей корректной обработки. В таких случаях система мониторинга должна выдавать своевременное предупреждение, а балансировщик нагрузки, реестр служб и другие подсистемы не должны направлять запросы на отказавший экземпляр.

Для решения этой задачи предназначен паттерн Health Check. Он предлагает определить для каждого сервиса конечную точку, которую можно использовать для проверки работоспособности, например /health. Этот API должен проверять статус хоста, подключение к другим сервисам, инфраструктуре и любую иную бизнес-логику. Клиент — служба мониторинга, реестр служб или балансировщик нагрузки — будет периодически обращаться к конечной точке для проверки работоспособности экземпляра сервиса.

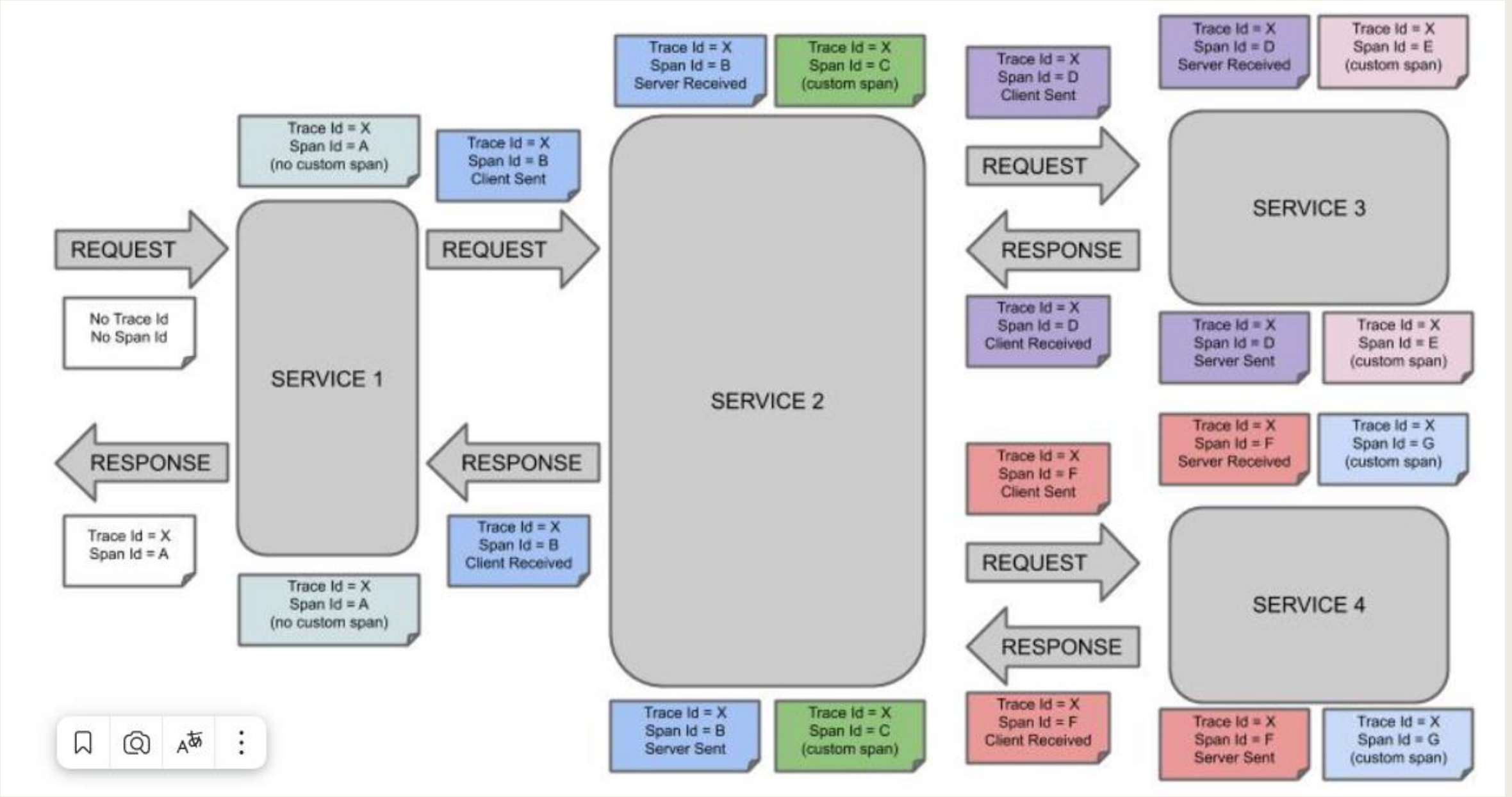
Шаблон «Распределенная трассировка» (Distributed Tracing)



В микросервисной архитектуре для выполнения клиентских запросов может потребоваться работа нескольких взаимосвязанных микросервисов. Каждый сервис обрабатывает запрос путем выполнения одной или нескольких операций, включая обращение к базе данных, публикацию сообщений и так далее. С увеличением числа сервисов становится все сложнее отследить место возникновения ошибок.

Паттерн Distributed Tracing разработан для решения этой проблемы. Он предлагает назначать каждому внешнему запросу уникальный идентификатор (TraceId), который будет передаваться всем сервисам, участвующим в обработке запроса, и фиксироваться в журналах. Это позволит разработчикам видеть, как обрабатывается отдельный запрос, путем поиска в агрегированных журналах его внешнего идентификатора.

Распределенная трассировка



Почему важно применять шаблоны проектирования в микросервисной архитектуре

- При переходе на микросервисы предстоит принимать множество архитектурных решений, от которых будет зависеть эффективность итогового продукта. Знание и правильный выбор подходящих паттернов во многом упрощают и ускоряют этот процесс. Ведь всегда лучше полагаться на многолетний опыт других разработчиков, чем пытаться изобрести собственное решение с нуля.
- Вот лишь некоторые преимущества, которых можно достичь, используя паттерны проектирования микросервисов:
  - Уменьшение ошибок при проектировании микросервисов — без необходимости их рефакторинга в дальнейшем.
  - Более быстрая и качественная миграция монолитов на микросервисную архитектуру.
  - Предотвращение ненужных вызовов и неэффективного использования ресурсов.
  - Отсутствие проблем с подключением новых сервисов, их интеграцией друг с другом и базами данных.
  - Лучшая масштабируемость: добавление дополнительных сервисов не вызывает затруднений в обслуживании зависимостей.
  - Повышение отказоустойчивости.
  - Минимизация угроз безопасности, в том числе сокрытие конечных точек микросервисов.
  - Сокращение работ по обслуживанию и отладке.

SOLID

- Есть такие принципы SOLID – про них очень любят спрашивать на собеседованиях.
- Принципы SOLID - это набор принципов, которые помогают разработчикам создавать гибкий, расширяемый и легко поддерживаемый код. SOLID - это акроним, каждая буква которого представляет собой один из принципов.
- Эти принципы помогают создавать гибкий и легко поддерживаемый код, который легко расширять и изменять. Они являются основой для принципов объектно-ориентированного программирования и широко применяются в разработке программного обеспечения.

SOLID расшифровка

- 1. Принцип единственной ответственности (Single Responsibility Principle, SRP) - класс должен иметь только одну причину для изменения. Это означает, что каждый класс должен быть ответственным только за одну часть функциональности.
- 2. Принцип открытости/закрытости (Open/Closed Principle, OCP) - программные сущности должны быть открыты для расширения, но закрыты для модификации. Это означает, что код должен быть написан таким образом, чтобы добавление новых функций происходило без изменения существующего кода.
- 3. Принцип подстановки Барбары Лисков (Liskov Substitution Principle, LSP) - объекты в программе должны быть заменяемыми на экземпляры их подтипов без изменения правильности выполнения программы. Это означает, что подклассы должны быть взаимозаменяемы с базовым классом.
- 4. Принцип разделения интерфейса (Interface Segregation Principle, ISP) - клиенты не должны зависеть от интерфейсов, которые они не используют. Это означает, что интерфейсы должны быть маленькими и специфичными для конкретных клиентов.
- 5. Принцип инверсии зависимостей (Dependency Inversion Principle, DIP) - модули верхнего уровня не должны зависеть от модулей нижнего уровня. Оба должны зависеть от абстракций. Это означает, что зависимости должны быть инвертированы, чтобы классы зависели от абстракций, а не от конкретных реализаций.

Микросервисы должны строиться по SOLID!

SOLID и хранение файлов

- Если в вашей системе необходимо хранить файлы (фотографии профилей, отчеты, документы и т.д.) для этого должен быть отдельный специальный компонент системы.
- Готовое решение – Amazon S3 и Minio.

Amazon S3

- Amazon S3 (Simple Storage Service) - это облачное хранилище данных, предоставляемое компанией Amazon Web Services (AWS).
- Оно позволяет пользователям хранить и извлекать любые объемы данных в любое время и из любого места в Интернете. S3 хранилище является высокомасштабируемым, гибким и надежным решением для хранения данных.
- Оно предоставляет возможность хранить файлы, аудио и видео контент, базы данных, архивы, резервные копии и другие типы данных. S3 также обеспечивает высокую доступность данных, автоматическое реплицирование данных на различные серверы и регионы, а также защиту данных с помощью шифрования и управления доступом. S3 хранилище используется в различных сферах, включая веб-хостинг, аналитику данных, резервное копирование и архивирование, разработку приложений и другие задачи, связанные с хранением и обработкой данных.

MiniO

MinIO — это решение для хранения объектов, которое предоставляет API-интерфейс Amazon Web Services, совместимый с S3, и поддерживает все основные функции S3. MinIO создан для развертывания где угодно — в публичном или частном облаке, в базовой инфраструктуре, в оркестрованных средах и периферийной инфраструктуре.

https://min.io/